Firebase aplicado a React Native en el proyecto Rotula

Ariel Alexander Hernández Merino (carné: HM192562)

Gerson Vladimir Borja Álvarez (carné: BA251704)

Universidad Don Bosco

DPS941 Diseño y Programación de Software Multiplataforma

Ing. Alexander Alberto Siguenza Campos

8 de noviembre de 2026

Firebase aplicado a React Native en el proyecto Rotula

Introducción

Rotula móvil es una aplicación para Android e iOS, hecha con React Native y Expo, que permite a tiendas y negocios pequeños crear los rótulos de sus ofertas y generar un PDF listo para imprimir, con cuatro rótulos por hoja. En el Avance 1, la aplicación dependía de la API web del proyecto: el inicio de sesión usaba una cookie y los datos se guardaban en MySQL.

El Avance 2 pide integrar Firebase Authentication y Firestore al proyecto, con acceso por correo electrónico y por Google, persistencia de los datos asociada al usuario autenticado, módulos secundarios funcionales y un historial de los registros almacenados. Este documento reúne la investigación sobre Firebase y explica cómo se aplicó en Rotula móvil.

El documento se divide en tres partes. El marco teórico explica qué es Firebase y cómo funcionan los servicios usados. Luego se describe la integración en la aplicación, incluidas las decisiones técnicas que exigió el cambio. Por último, se presentan las pruebas realizadas y las conclusiones. El código del proyecto está disponible en https://github.com/Slincker5/rotula-movil.

Marco teórico

Qué es Firebase

Firebase es una plataforma de Google que ofrece servicios de backend listos para usarse desde una aplicación, como autenticación y bases de datos, junto con los SDK para consumirlos. De este modo, el equipo no tiene que crear ni mantener su propio servidor para esas funciones. Esa ventaja tiene un costo: el rendimiento también depende de la red. Kot y Smołka (2023) compararon Realtime Database y Cloud Firestore con SQLite en dispositivos móviles, y encontraron que Realtime Database superó a Cloud Firestore en sus pruebas.

Firebase Authentication

Firebase Authentication proporciona servicios de backend, SDK y bibliotecas de interfaz para identificar a los usuarios de una aplicación. Admite correo y contraseña, proveedores federados como Google, Apple, Facebook, Twitter y GitHub, número de teléfono, sistemas de

autenticación propios y acceso anónimo, y se apoya en OAuth 2.0 y OpenID Connect (Google, s.f.-e).

Observador de la sesión

La forma recomendada de conocer al usuario actual es registrar un observador con `onAuthStateChanged`. Así se evita consultar el objeto `Auth` mientras todavía se está inicializando, momento en el que `currentUser` puede ser nulo (Google, s.f.-f). La sesión puede conservarse de tres formas: `local`, `session` o `none`. La persistencia `local` se mantiene aunque se cierre la aplicación y es la predeterminada en React Native, por lo que solo termina con un cierre de sesión explícito (Google, s.f.-b).

Acceso con Google

Además del flujo completo que ofrece el SDK, Firebase permite un flujo manual: otra biblioteca realiza el acceso con Google y la aplicación intercambia el ID token obtenido por una credencial de Firebase, con la que se autentica mediante `signInWithCredential` (Google, s.f.-a).

Cloud Firestore

Cloud Firestore es una base de datos NoSQL sin servidor, pensada para que las aplicaciones cliente trabajen directamente con ella y reciban notificaciones en tiempo real (Kesavan et al., 2023). Su unidad de almacenamiento es el documento, un conjunto de campos parecido a JSON. Los documentos se agrupan en colecciones, y una colección solo contiene documentos. Cada documento puede tener subcolecciones, con un anidamiento máximo de 100 niveles (Google, s.f.-d).

Este modelo cambia la forma de pensar los datos respecto de MySQL. No hay tablas ni columnas fijas, y tampoco uniones entre tablas, de modo que las relaciones se representan con la propia ruta del documento o con campos que guardan identificadores. La Tabla 1 muestra la equivalencia usada en Rotula.

Tabla 1

Equivalencia entre MySQL y Firestore en Rotula móvil

MySQL (Avance 1)	Firestore (Avance 2)
Tabla `usuarios`	Colección `usuarios`, con un documento por uid
Tabla `rotulos` con columna `usuario_id`	Subcolección `usuarios/{uid}/rotulos`
Tabla `documentos` con columna `usuario_id`	Subcolección `usuarios/{uid}/documentos`
Tabla `productos`	Subcolección `usuarios/{uid}/productos`
Llave foránea `documento_id`	Campo `documento_id` con el identificador del documento

Escrituras en lote

Un lote es un conjunto de escrituras sobre uno o varios documentos que se completa de forma atómica: o todas las operaciones se aplican, o no se aplica ninguna (Google, s.f.-g). Esta garantía sustituye a las transacciones que antes hacía el servidor con MySQL.

Límites del plan gratuito

La cuota gratuita de Firestore incluye 1 GiB de almacenamiento, 50,000 lecturas, 20,000 escrituras y 20,000 borrados de documentos por día, y 10 GiB de transferencia saliente por mes. Cada documento puede pesar hasta 1 MiB (Google, s.f.-h). Estos límites son suficientes para un proyecto académico y explican por qué los PDF no se guardan en la base de datos.

Reglas de seguridad

Como la aplicación accede directamente a Firestore, la protección de los datos no puede depender del código del cliente. Las reglas de seguridad se evalúan en los servidores de Firebase y definen quién puede leer o escribir cada ruta. El modo bloqueado niega todo acceso, y el patrón de contenido por dueño solo lo permite cuando `request.auth.uid` coincide con el identificador de la ruta (Google, s.f.-c).

Firebase en React Native con Expo

Expo describe dos caminos. El SDK de JavaScript cubre Authentication, Firestore, Realtime Database y Storage, y funciona en Expo Go, pero no incluye Analytics, Dynamic Links ni Crashlytics. React Native Firebase envuelve los SDK nativos y no puede usarse en Expo Go porque requiere código nativo propio (Expo, 2026b). Rotula solo necesita Authentication y Firestore, por lo que se eligió el SDK de JavaScript.

El acceso con Google es la excepción. La biblioteca usada se apoya en los SDK nativos de Google (React Native Google Sign In, s.f.), de modo que requiere un development build: una versión propia de Expo Go que admite bibliotecas nativas (Expo, 2026a). Además, el ID token solo se obtiene si se configura el `webClientId` (React Native Google Sign In, s.f.).

Integración en Rotula móvil

Autenticación

El contexto de sesión de la aplicación dejó de llamar a la API web y ahora se suscribe a `onAuthStateChanged`. La Figura 1 resume ese cambio. La navegación no necesitó cambios en su lógica: sigue mostrando Login y Registro sin sesión, y las pestañas principales con sesión. Al entrar por primera vez, la aplicación crea el perfil del usuario en `usuarios/{uid}`. Con correo usa los datos del formulario de registro y con Google, el nombre de la cuenta. La persistencia se configuró con AsyncStorage para que la sesión siga abierta al cerrar la aplicación.

Figura 1

Observador de la sesión en el contexto de la aplicación

```
useEffect(() =>
  onAuthStateChanged(auth, async (usuarioAuth) => {
    if (!usuarioAuth) {
      setUsuario(null);
      return;
    }
    const perfil = await obtenerPerfil(usuarioAuth, datosRegistro.current);
    setUsuario(perfil);
  }), []);
```

Nota. Fragmento simplificado de `src/contexto/Sesion.js`.

Persistencia de los datos

Todo lo que genera un usuario se guarda debajo de `usuarios/{uid}`, tal como muestra la Tabla 1. Las funciones de acceso a datos se agruparon en la carpeta `src/firebase`, de modo que las pantallas no conocen los detalles de Firestore. Los errores de Firebase se traducen a mensajes en español antes de llegar a la interfaz.

Las reglas de seguridad aplican el patrón de contenido por dueño y, además, validan los datos. Por ejemplo, un rótulo debe tener precios mayores que cero y una fecha final que no sea anterior a la inicial. Una vez que un rótulo sale en un PDF, ya no se puede modificar ni borrar, para que el documento siempre pueda volver a generarse. La Figura 2 muestra un fragmento de esas reglas.

Figura 2

Regla de seguridad para los rótulos de un usuario

```
match /usuarios/{uid}/rotulos/{id} {
  allow read: if esDueno(uid);
  allow create: if esDueno(uid)
    && rotuloValido(request.resource.data)
    && request.resource.data.documento_id == null;
  allow delete: if esDueno(uid) && resource.data.documento_id == null;
}
```

Nota. Fragmento adaptado de `firestore.rules`.

Generación del PDF

Este fue el cambio más delicado. El PDF lo generaba la API web a partir de MySQL, por lo que dejaría de funcionar con los datos en Firestore. La lógica de dibujo se llevó a la aplicación con la biblioteca pdf-lib, y la plantilla se incluyó dentro del código. Al generar un documento, la aplicación pinta primero el PDF y luego usa un lote que registra el documento y vincula sus rótulos. Si algo falla, no queda un documento a medias. El archivo no se guarda: el historial conserva los datos y, al abrirlo, el PDF se vuelve a pintar con los mismos rótulos.

Módulos secundarios e historial

Se agregó el módulo Mis productos, un catálogo por usuario para crear, listar, editar y borrar productos. Al crear un afiche, el usuario puede elegir un producto y el formulario se llena con su código de barras, descripción y precio normal. También se agregó la edición del perfil. El historial lee la subcolección de documentos del usuario y permite volver a abrir cualquier PDF. La pantalla de inicio calcula sus resúmenes con esos mismos registros. La Tabla 2 relaciona cada requerimiento del avance con su implementación.

Tabla 2

Requerimientos del Avance 2 e implementación en Rotula móvil

Requerimiento	Implementación	Ubicación
Autenticación con correo y Google	Firebase Authentication con observador de sesión	`src/contexto/Sesion.js`, `src/firebase/google.js`
Persistencia por usuario	Subcolecciones bajo `usuarios/{uid}` y reglas de seguridad	`src/firebase/`, `firestore.rules`
Módulos secundarios	Mis productos y edición de perfil	`src/pantallas/Productos.js`, `src/pantallas/Perfil.js`
Historial de registros	Lista de documentos y nueva generación del PDF	`src/pantallas/Historial.js`

Pruebas

Para no crear cuentas de prueba en el proyecto real, los flujos se verificaron con Firebase Local Emulator Suite y con las mismas reglas publicadas en el proyecto. En la versión web de la aplicación se comprobó el registro por correo, la creación del perfil, el catálogo de productos, la elección de un producto al crear un afiche, la generación del documento, el historial y la edición del perfil. También se comprobó que una petición sin sesión y otra de un usuario distinto reciben una respuesta 403 al intentar acceder a datos ajenos.

Las pruebas detectaron un error que no aparecía al compilar: la versión ESM de la biblioteca tslib impedía que la aplicación cargara en Metro. Se resolvió con un archivo

`metro.config.js` que dirige esa dependencia a su versión UMD. El caso muestra que compilar sin errores no garantiza que una aplicación funcione y que es necesario ejecutarla.

Conclusiones

Firebase permitió sustituir la autenticación propia y la base de datos MySQL por servicios administrados, sin tener que mantener un servidor. La mayor parte del esfuerzo no estuvo en conectar el SDK, sino en replantear lo que antes resolvía el backend: las reglas de seguridad reemplazaron los filtros por `usuario_id`, los lotes reemplazaron las transacciones del servidor y la generación del PDF pasó al dispositivo.

El cambio también tiene costos. La cuenta móvil ya no se comparte con la versión web, que sigue usando MySQL, y el acceso con Google obliga a usar un development build en lugar de Expo Go. Para un proyecto de este tamaño, esos costos son aceptables a cambio de una aplicación que ya no depende de un servidor propio para funcionar.

Referencias

Expo. (2026a, 2 de noviembre). *Introduction to development builds*. Expo Documentation. https://docs.expo.dev/develop/development-builds/introduction/

Expo. (2026b, 2 de noviembre). *Using Firebase*. Expo Documentation. https://docs.expo.dev/guides/using-firebase/

Google. (s.f.-a). *Authenticate using Google with JavaScript*. Firebase. Recuperado el 6 de noviembre de 2026, de https://firebase.google.com/docs/auth/web/google-signin

Google. (s.f.-b). *Authentication state persistence*. Firebase. Recuperado el 6 de noviembre de 2026, de https://firebase.google.com/docs/auth/web/auth-state-persistence

Google. (s.f.-c). *Basic security rules*. Firebase. Recuperado el 6 de noviembre de 2026, de https://firebase.google.com/docs/rules/basics

Google. (s.f.-d). *Cloud Firestore data model*. Firebase. Recuperado el 6 de noviembre de 2026, de https://firebase.google.com/docs/firestore/data-model

Google. (s.f.-e). *Firebase Authentication*. Firebase. Recuperado el 6 de noviembre de 2026, de https://firebase.google.com/docs/auth

Google. (s.f.-f). *Manage users in Firebase*. Firebase. Recuperado el 6 de noviembre de 2026, de https://firebase.google.com/docs/auth/web/manage-users

Google. (s.f.-g). *Transactions and batched writes*. Firebase. Recuperado el 6 de noviembre de 2026, de https://firebase.google.com/docs/firestore/manage-data/transactions

Google. (s.f.-h). *Usage and limits*. Firebase. Recuperado el 6 de noviembre de 2026, de https://firebase.google.com/docs/firestore/quotas

Kesavan, R., Gay, D., Thevessen, D., Shah, J., & Mohan, C. (2023). Firestore: The NoSQL serverless database for the application developer. En *2023 IEEE 39th International Conference on Data Engineering (ICDE)* (pp. 3367-3379). IEEE. https://research.google/pubs/pub52292/

Kot, S., & Smołka, J. (2023). A performance analysis of a cloud database on mobile devices. *Journal of Computer Sciences Institute*, *29*, 360-365. https://doi.org/10.35784/jcsi.3798

React Native Google Sign In. (s.f.). *Original Google sign in*. Recuperado el 6 de noviembre de 2026, de https://react-native-google-signin.github.io/docs/original